

## Dimensionality Reduction using PCA

- there are two types of techniques to reduce dimension of a data

1) projections e.g., PCA, LDA etc.

2) manifold based techniques e.g., t-SNE, TDA etc.

Start coding or [generate](#) with AI.

### IRIS Dataset

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.decomposition import PCA
```

```
sns.get_dataset_names()
```

```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic']
```

```
iris = sns.load_dataset("iris").dropna()
iris.head()
```

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

Next steps: [Generate code with iris](#) [New interactive sheet](#)

```
X=iris.iloc[:,0:4]
y=iris.iloc[:, -1]
print(X)
print(y)
```

|     | sepal_length | sepal_width | petal_length | petal_width |
|-----|--------------|-------------|--------------|-------------|
| 0   | 5.1          | 3.5         | 1.4          | 0.2         |
| 1   | 4.9          | 3.0         | 1.4          | 0.2         |
| 2   | 4.7          | 3.2         | 1.3          | 0.2         |
| 3   | 4.6          | 3.1         | 1.5          | 0.2         |
| 4   | 5.0          | 3.6         | 1.4          | 0.2         |
| ..  | ...          | ...         | ...          | ...         |
| 145 | 6.7          | 3.0         | 5.2          | 2.3         |
| 146 | 6.3          | 2.5         | 5.0          | 1.9         |
| 147 | 6.5          | 3.0         | 5.2          | 2.0         |
| 148 | 6.2          | 3.4         | 5.4          | 2.3         |
| 149 | 5.9          | 3.0         | 5.1          | 1.8         |

```
[150 rows x 4 columns]
```

```
0      setosa
1      setosa
2      setosa
3      setosa
4      setosa
```

```

    ...
145  virginica
146  virginica
147  virginica
148  virginica
149  virginica

```

```
Name: species, Length: 150, dtype: object
```

X

|     | sepal_length | sepal_width | petal_length | petal_width |
|-----|--------------|-------------|--------------|-------------|
| 0   | 5.1          | 3.5         | 1.4          | 0.2         |
| 1   | 4.9          | 3.0         | 1.4          | 0.2         |
| 2   | 4.7          | 3.2         | 1.3          | 0.2         |
| 3   | 4.6          | 3.1         | 1.5          | 0.2         |
| 4   | 5.0          | 3.6         | 1.4          | 0.2         |
| ... | ...          | ...         | ...          | ...         |
| 145 | 6.7          | 3.0         | 5.2          | 2.3         |
| 146 | 6.3          | 2.5         | 5.0          | 1.9         |
| 147 | 6.5          | 3.0         | 5.2          | 2.0         |
| 148 | 6.2          | 3.4         | 5.4          | 2.3         |
| 149 | 5.9          | 3.0         | 5.1          | 1.8         |

150 rows  $\times$  4 columns

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()
target=encoder.fit_transform(y)
target
```

[illegible]

```
pca = PCA(n_components=4)
pca.fit(X)
print('sum(pca.explained_variance_)',sum(pca.explained_variance_ratio_))
print('pca.explained_variance_',pca.explained_variance_)
#print('pca.singular_values_',pca.singular_values_)
pc=pca.fit_transform(X)
print(pc.shape)

pc_df=pd.DataFrame(pc,columns=['PC1','PC2','PC3','PC4'])
pc_df["species"]=y
print(pc_df)
```

```
sum(pca.explained_variance_) 1.0
pca.explained_variance_ [4.22824171 0.24267075 0.0782095 0.02383509]
(150, 4)
   PC1      PC2      PC3      PC4  species
0 -2.684126  0.319397 -0.027915  0.002262  setosa
1 -2.714142 -0.177001 -0.210464  0.099027  setosa
2 -2.888991 -0.144949  0.017900  0.019968  setosa
3 -2.745343 -0.318299  0.031559 -0.075576  setosa
4 -2.728717  0.326755  0.090079 -0.061259  setosa
..    ...      ...      ...      ...    ...
145 1.944110  0.187532  0.177825  0.426196  virginica
146 1.527167 -0.375317 -0.121898  0.254367  virginica
147 1.764346  0.078859  0.130482  0.137001  virginica
148 1.900942  0.116628  0.723252  0.044595  virginica
149 1.390189 -0.282661  0.362910 -0.155039  virginica

[150 rows x 5 columns]
```

```
pca.explained_variance_[0]/sum(pca.explained_variance_)
```

```
np.float64(0.9246187232017327)
```

Start coding or [generate](#) with AI.

```
pca.components_
```

```
array([[ 0.36138659, -0.08452251,  0.85667061,  0.3582892 ],
       [ 0.65658877,  0.73016143, -0.17337266, -0.07548102],
       [-0.58202985,  0.59791083,  0.07623608,  0.54583143],
       [ 0.31548719, -0.3197231 , -0.47983899,  0.75365743]])
```

```
import numpy as np
np.round(pca.components_, 3)
```

```
array([[ 0.361, -0.085,  0.857,  0.358],
       [ 0.657,  0.73 , -0.173, -0.075],
       [-0.582,  0.598,  0.076,  0.546],
       [ 0.315, -0.32 , -0.48 ,  0.754]])
```

## ✧ Project data points on first two PCAs

and then reconstruct them

```
U=pca.components_[ :2, :]
```

## ✧ Regenerate Original Point

```
import numpy as np
#X_first=np.array(X.iloc[0,:])
X_first=np.array([15.1,3.5,160.0,500.0])
print(X_first)
```

```
new_x=np.matmul(U,X_first)
print(new_x)
mul=np.matmul(U.T,new_x)
print(mul)
```

```
[ 15.1   3.5 160.   500. ]
[321.37300426 -53.01008054]
[ 81.33407106 -65.86917074 284.50130512 119.14573063]
```

```
np.linalg.norm(X_first - mul)
```

```
np.float64(412.00677602606044)
```

## ▼ Detect Anomalies using Percentiles

```
# We will assume U contains the first 2 principal components
# and X is the original data.

# To project, multiply X by U.T (transpose of U), which reduces dimensionality
X_projected = np.dot(X, U.T)

# Step 4: Reconstruct the data from the projected points
# Reconstructing is simply multiplying the projected data with the transpose of U.
X_reconstructed = np.dot(X_projected, U)

# Step 5: Calculate the reconstruction errors (Euclidean distance)
reconstruction_errors = np.linalg.norm(X.values - X_reconstructed, axis=1)

# Step 6: Calculate the 95th percentile of the reconstruction errors
percentile_95 = np.percentile(reconstruction_errors, 95)

# Output the reconstruction errors and the 95th percentile threshold
print("Reconstruction Errors:\n", reconstruction_errors)
print("\n95th Percentile of Reconstruction Errors:", percentile_95)

# Optionally, you can flag anomalies by comparing reconstruction errors to the 95th percentile
anomalies = X[reconstruction_errors > percentile_95]
print("\nDetected Anomalies:\n", anomalies)
```

Reconstruction Errors:

```
[0.66049963 0.84487342 0.61409831 0.61009614 0.54997237 0.46673701
0.38273738 0.65844615 0.61404105 0.83431571 0.70882314 0.56265666
0.83971043 0.57320088 0.79352475 0.35508862 0.51163303 0.60680192
0.75218462 0.42989049 0.87264127 0.42346574 0.34068138 0.60378259
0.60681331 0.88527365 0.53748263 0.71193491 0.7784375 0.61166315
0.71525479 0.80226539 0.51384465 0.49120216 0.77503389 0.80660728
0.91150437 0.57464908 0.56066156 0.71349139 0.55791813 1.02209789
0.4524302 0.39474164 0.39711735 0.73770166 0.51582867 0.55594545
0.65387032 0.72114791 1.03852658 0.65069337 0.97080978 0.81367524
0.94289813 0.65226669 0.4766034 0.6216667 1.04969454 0.35286983
0.90175258 0.50160572 1.32621373 0.72300599 0.54913373 0.94880358
0.37583588 0.93621482 1.15330248 0.87008709 0.20349619 0.86635712
0.98024477 0.92090707 0.95369037 0.9504175 1.1535572 0.79761468
0.59975293 0.95123772 0.87646786 0.93982033 0.80909559 0.65105075
0.34054111 0.28419893 0.86902235 1.25240038 0.47863262 0.69187199
0.7227422 0.66844194 0.86141949 0.73764578 0.62925622 0.61229494
0.57776599 0.83903456 0.67453152 0.62699564 0.14268434 0.33903975
0.74334792 0.58902799 0.34808112 0.99106222 0.15721278 1.09147333
0.98653832 0.2224993 0.39916344 0.67098933 0.61241986 0.36596854
0.29621347 0.24055168 0.61793674 0.60441467 1.14580378 0.98005125
0.43985956 0.12313204 1.21747244 0.70834886 0.35818212 0.88109277
0.59701975 0.40166817 0.47806956 1.11334997 1.13212125 0.79940946
0.43358926 0.81651273 0.96844075 1.00617544 0.10538885 0.53421934
0.35025414 0.63001539 0.39403397 0.70095624 0.33903975 0.34017103
0.24367754 0.6003896 0.78548606 0.51197285 0.09208783 0.32836153]
```

95th Percentile of Reconstruction Errors: 1.1035054854169142

```

Detected Anomalies:
      sepal_length  sepal_width  petal_length  petal_width
62              6.0          2.2          4.0          1.0
68              6.2          2.2          4.5          1.5
76              6.8          2.8          4.8          1.4
87              6.3          2.3          4.4          1.3
118             7.7          2.6          6.9          2.3
122             7.7          2.8          6.7          2.0
129             7.2          3.0          5.8          1.6
130             7.4          2.8          6.1          1.9

```

## ▼ Anomalies using Standard Deviations (2SD and 3SD)

Double-click (or enter) to edit

```

# Project the data onto the principal components (U)
X_projected = np.dot(X, U.T)

# Reconstruct the data from the projected points
X_reconstructed = np.dot(X_projected, U)

# Calculate the reconstruction errors (Euclidean distance)
reconstruction_errors = np.linalg.norm(X.values - X_reconstructed, axis=1)

# Compute the 98th percentile of the reconstruction errors
percentile_98 = np.percentile(reconstruction_errors, 98)

# Output the 98th percentile threshold
print("98th Percentile of Reconstruction Errors:", percentile_98)

# Optionally, use standard deviation-based threshold (Mean + 2 × Std)
mean_error = np.mean(reconstruction_errors)
std_error = np.std(reconstruction_errors)
threshold_2sd = mean_error + 2 * std_error
threshold_3sd = mean_error + 3 * std_error

print(f"Threshold based on 2 standard deviations: {threshold_2sd}")
print(f"Threshold based on 3 standard deviations: {threshold_3sd}")

# Flag anomalies by comparing reconstruction errors to the new thresholds
anomalies_percentile = X[reconstruction_errors > percentile_98]
anomalies_2sd = X[reconstruction_errors > threshold_2sd]
anomalies_3sd = X[reconstruction_errors > threshold_3sd]

print("\nDetected anomalies using 98th percentile:\n", anomalies_percentile)
print("\nDetected anomalies using 2 standard deviations threshold:\n", anomalies_2sd)
print("\nDetected anomalies using 3 standard deviations threshold:\n", anomalies_3sd)

```

```

98th Percentile of Reconstruction Errors: 1.154835502572664
Threshold based on 2 standard deviations: 1.175191151907255
Threshold based on 3 standard deviations: 1.4329001769823486

```

```

Detected anomalies using 98th percentile:
      sepal_length  sepal_width  petal_length  petal_width
62              6.0          2.2          4.0          1.0
87              6.3          2.3          4.4          1.3
122             7.7          2.8          6.7          2.0

```

```

Detected anomalies using 2 standard deviations threshold:
      sepal_length  sepal_width  petal_length  petal_width
62              6.0          2.2          4.0          1.0
87              6.3          2.3          4.4          1.3
122             7.7          2.8          6.7          2.0

```

```

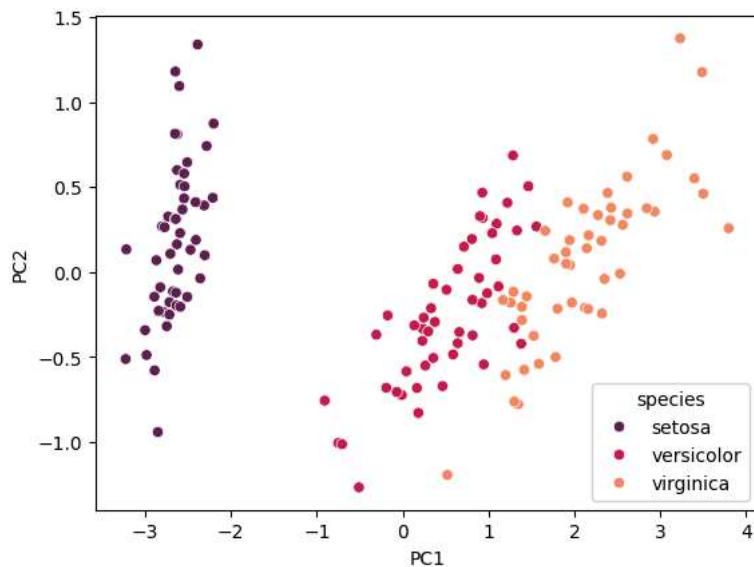
Detected anomalies using 3 standard deviations threshold:

```

```
Empty DataFrame
Columns: [sepal_length, sepal_width, petal_length, petal_width]
Index: []
```

```
sns.scatterplot(data=pc_df, x="PC1", y="PC2", hue="species", palette="rocket")
```

```
<Axes: xlabel='PC1', ylabel='PC2'>
```



## ▼ MNIST Dataset

```
from keras.datasets import mnist
```

```
len(mnist.load_data())
```

```
2
```

```
type(mnist.load_data())
```

```
tuple
```

```
A=[1,5,9]
A[1]=6
print(A)
```

```
[1, 6, 9]
```

```
len(mnist.load_data()[0])
```

```
2
```

```
len(mnist.load_data()[1])
```

```
2
```

```
[[[],[]],[[],[]]]
```

```
mnist.load_data()[0][1].shape
```

```
(60000,)
```

```
(X_train, y_train), (X_test,y_test) = mnist.load_data()
```

```
print(X_test.shape)
print(y_test.shape)
```

```
(10000, 28, 28)
(10000,)
```

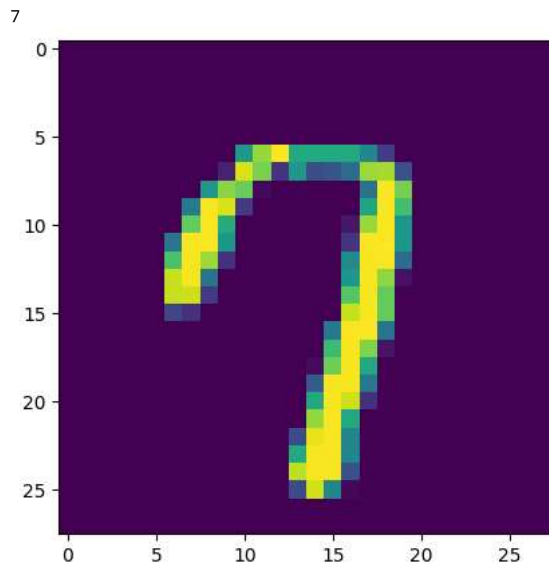
```
print(X_test[0])
```

```
[[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  84 185 159 151 60 36  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 198 198 170 52  0  0 222 254 254 254 254 241 198 198 198 198 198
   198 198 170 52  0  0  0  0  0  0]
 [ 0  0  0  0  0  0 67 114 72 114 163 227 254 225 254 254 254 250
   229 254 254 140  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  17 66 14 67 67 67 59
   21 236 254 106  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   83 253 209 18  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  22
   233 255 83  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 129
   254 238 44  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  59 249
   254 62  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  133 254
   187 5  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  9 205 248
   58  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  126 254 182
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  75 251 240 57
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  19 221 254 166  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  3 203 254 219 35  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  38 254 254 77  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  31 224 254 115  1  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  133 254 254 52  0  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  61 242 254 254 52  0  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  121 254 254 219 40  0  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  121 254 207 18  0  0  0  0
   0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0]]
```

```
X_test[15].shape
```

```
(28, 28)
```

```
plt.imshow(X_test[1055])  
print(y_test[1055])
```



```
X_test.shape
```

```
(10000, 28, 28)
```

```
Z=X_test.reshape(10000,28*28)
```

```
from sklearn.preprocessing import StandardScaler # to normalize data  
standardized_data = StandardScaler().fit_transform(Z)
```

```
standardized_data[0]
```



```

-5.90584640e-01, -4.74233049e-01, -3.68789557e-01, -2.74082099e-01,
-1.74264813e-01, -6.96188843e-02, -1.80031510e-02, 0.00000000e+00,
0.00000000e+00, 0.00000000e+00, -1.68610568e-02, -4.51688568e-02,
-1.31668459e-01, -2.67838929e-01, -3.98906806e-01, -5.48202377e-01,
-6.90077015e-01, -7.89823563e-01, -8.31599129e-01, -8.61314493e-01,
-9.56815660e-01, -1.11036634e+00, -1.22743073e+00, -1.31006468e+00,
-2.57368600e-02, 1.14239899e+00, 7.61423491e-01, -7.06825874e-01,
-6.08999426e-01, -4.92457882e-01, -3.80502867e-01, -2.79282191e-01,
-1.73984018e-01, -7.67235054e-02, -1.95871373e-02, -1.00005000e-02,
0.00000000e+00, -1.00005000e-02, -2.48178080e-02, -5.52275065e-02,
-1.48243512e-01, -2.83202341e-01, -4.02212500e-01, -5.34598048e-01,
-6.56007943e-01, -7.38083794e-01, -7.81657503e-01, -8.24620535e-01,
-9.18824463e-01, -1.04078449e+00, -1.13391454e+00, -1.09212795e+00,
7.05920310e-01, 1.17679031e+00, -3.73781820e-01, -7.58547572e-01,
-6.28680640e-01, -5.01492113e-01, -3.81043892e-01, -2.70505206e-01,
-1.68251255e-01, -7.84168728e-02, -2.27999680e-02, -1.57856413e-02,
0.00000000e+00, 0.00000000e+00, -2.69850288e-02, -6.76999793e-02,
-1.67498207e-01, -2.98089736e-01, -4.11096027e-01, -5.22810883e-01,
-6.25838621e-01, -6.93423683e-01, -7.31704263e-01, -7.67086709e-01,
-8.29980030e-01, -9.21590434e-01, -1.00562716e+00, 7.79492952e-02,
1.22959017e+00, 6.36500653e-01, -9.01400043e-01, -7.69630793e-01,
-6.35363773e-01, -4.94618472e-01, -3.69117095e-01, -2.55794246e-01,
-1.56732083e-01, -7.83809414e-02, -2.67109338e-02, -1.48726634e-02,
0.00000000e+00, -1.00005000e-02, -3.48385687e-02, -8.69311199e-02,

```

```

pcaa = PCA(n_components=2)
principalComponents = pcaa.fit_transform(standardized_data)
principal_df = pd.DataFrame(data = principalComponents
                             , columns = ['PC1', 'PC2'])
principal_df

```

|      | PC1       | PC2       |
|------|-----------|-----------|
| 0    | -5.458311 | 6.414007  |
| 1    | 2.804432  | -8.028850 |
| 2    | -7.411241 | -3.864044 |
| 3    | 8.751242  | 0.045991  |
| 4    | -0.065756 | 6.296290  |
| ...  | ...       | ...       |
| 9995 | 3.579478  | -3.367965 |
| 9996 | 3.244816  | -1.981771 |
| 9997 | -5.004352 | 1.153646  |
| 9998 | -1.420854 | -4.423081 |
| 9999 | 13.142683 | -4.262803 |

10000 rows × 2 columns

```

principal_df["labels"]=y_test
principal_df

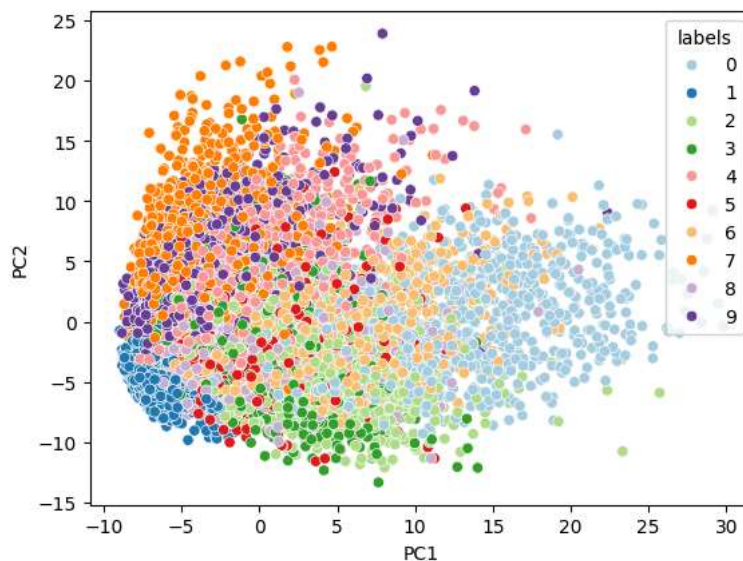
```

|      | PC1       | PC2       | labels |
|------|-----------|-----------|--------|
| 0    | -5.458311 | 6.414007  | 7      |
| 1    | 2.804432  | -8.028850 | 2      |
| 2    | -7.411241 | -3.864044 | 1      |
| 3    | 8.751242  | 0.045991  | 0      |
| 4    | -0.065756 | 6.296290  | 4      |
| ...  | ...       | ...       | ...    |
| 9995 | 3.579478  | -3.367965 | 2      |
| 9996 | 3.244816  | -1.981771 | 3      |
| 9997 | -5.004352 | 1.153646  | 4      |
| 9998 | -1.420854 | -4.423081 | 5      |
| 9999 | 13.142683 | -4.262803 | 6      |

10000 rows × 3 columns

```
sns.scatterplot(data=principal_df, x="PC1", y="PC2", palette="Paired", hue="labels")
```

```
<Axes: xlabel='PC1', ylabel='PC2'>
```



```
print(sum(pcaa.explained_variance_ratio_))
print(pcaa.explained_variance_ratio_)
print(pcaa.singular_values_)
```

```
0.10440571415976757
[0.06196825 0.04243746]
[643.38786188 532.4304929 ]
```

## ✧ tSNE: t-distributed Stochastic Neighbor Embedding

MNIST dataset

## ✧ t- distributed stochastic neighbor embedding

```
from sklearn.manifold import TSNE
tsne = TSNE(n_components = 2, random_state = 42)
X_reduced = tsne.fit_transform(Z)
```

```
X_reduced=pd.DataFrame(X_reduced,columns=["Dim1","Dim2"])
X_reduced["labels"]=y_test
```

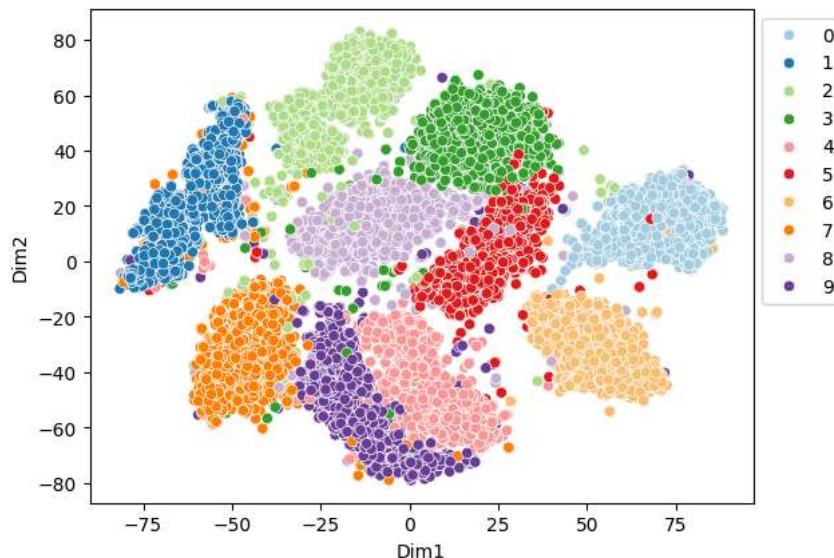
X\_reduced

|      | Dim1       | Dim2       | labels |
|------|------------|------------|--------|
| 0    | -50.220901 | -29.904476 | 7      |
| 1    | -34.204624 | 41.985683  | 2      |
| 2    | -57.534454 | 23.979694  | 1      |
| 3    | 59.987061  | 10.608885  | 0      |
| 4    | 15.988505  | -51.244221 | 4      |
| ...  | ...        | ...        | ...    |
| 9995 | -21.661270 | 46.930294  | 2      |
| 9996 | 33.840553  | 44.071186  | 3      |
| 9997 | -6.769926  | -27.899359 | 4      |
| 9998 | 8.319714   | -14.010859 | 5      |
| 9999 | 51.947758  | -20.907963 | 6      |

10000 rows × 3 columns

```
plt.figure()
sns.scatterplot(data=X_reduced, x="Dim1", y="Dim2", hue="labels",palette="Paired")
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
```

<matplotlib.legend.Legend at 0x7d47e82a1dc0>



Double-click (or enter) to edit

Start coding or [generate](#) with AI.

